



Unit 1: Introduction

Dr. K Sowjanya, Dr. Rashmi Maheshwari^a,

Assistant Professor
CSED, MNNIT Allahabad

Email 1: sowjanya.k@mnnit.ac.in Email 2: m.rashmi@mnnit.ac.in

Department of Computer Science and Engineering
MNNIT Allahabad, Prayagraj | , 2026



1. History of Java
2. Features of Java
3. Meaning of Object Orientation
4. Object Identity
5. Encapsulation
6. Information Hiding
7. Polymorphism
8. Generality
9. Importance of Modeling
10. Principles of Modeling



Early Vision

- ▶ Java was initiated in **June 1991** at **Sun Microsystems** by **James Gosling, Mike Sheridan, and Patrick Naughton**.
- ▶ Developed by the **Green Team** for embedded electronic devices such as set-top boxes and interactive television.
- ▶ Although initially intended for digital TV, Java became ideal for **Internet programming**.

Key Milestones

- ▶ Designed with principles of **simplicity, portability, platform independence, security, high performance, multithreading, and object-oriented programming**.
- ▶ Publicly released in **1995**; **Netscape** integrated Java into its web browser, accelerating its popularity.
- ▶ **Time Magazine** listed Java among the **Ten Best Products of 1995**.
- ▶ **JDK 1.0** was officially released on **January 23, 1996**.

Naming History

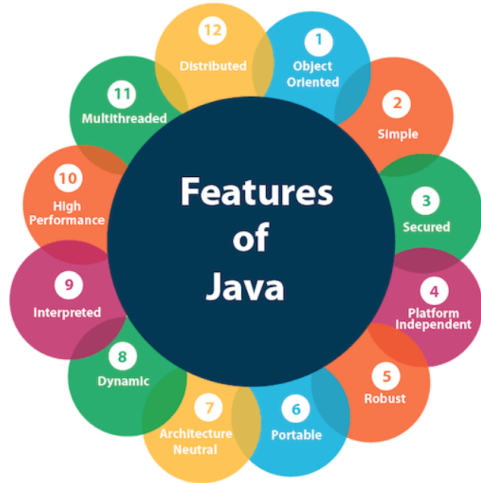
- ▶ Originally named **Greentalk** (.gt), later renamed **Oak**.
- ▶ In 1995, renamed **Java** because “Oak” was already trademarked.
- ▶ Other names considered included *Dynamic*, *Revolutionary*, *Silk*, *Jolt*, and *DNA*.
- ▶ The name **Java** was inspired by **Java coffee** from the Indonesian island of Java and is **not an acronym**.



Features of Java



- ▶ Simple
- ▶ Object-Oriented
- ▶ Portable
- ▶ Platform Independent
- ▶ Secured
- ▶ Robust
- ▶ Architecture Neutral
- ▶ Interpreted
- ▶ High Performance
- ▶ Multithreaded
- ▶ Distributed
- ▶ Dynamic





Why did OOP come into existence?



Programming Evolution

► POP $\Rightarrow$ OOP

Learning Goal

► Understand the Need for OOP



Why did OOP come into existence?

Programming Evolution

► POP $\Rightarrow$ OOP

Learning Goal

► Understand the Need for OOP

Scenario:

Imagine developing software for an entire hospital, banking system, or social media platform using only functions. Would it remain manageable as the software grows?



Why did OOP come into existence?

Programming Evolution

► POP $\Rightarrow$ OOP

Learning Goal

► Understand the Need for OOP

Scenario:

Imagine developing software for an entire hospital, banking system, or social media platform using only functions. Would it remain manageable as the software grows?

What is POP?

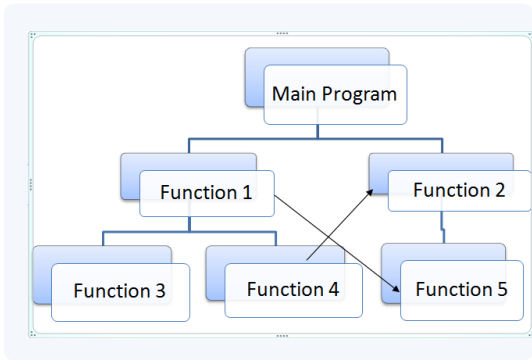
Problems with POP

Why OOP?

Real-World example



What is POP



Procedure-Oriented Programming (POP)

- ▶ Software is organized around **functions (procedures)**.
- ▶ Data is secondary and shared among functions.
- ▶ Examples: **C, Pascal, FORTRAN**.
- ▶ Best suited for small and medium-sized programs.



Challenges of Procedure-Oriented Programming

- ▶ Difficult to maintain very large software.
- ▶ Limited code reuse.
- ▶ Data is easily accessible by multiple functions, reducing security.
- ▶ Higher development and maintenance cost.
- ▶ Poor representation of real-world objects.

PROBLEMS with Procedural Programming





Why OOP?



Object-Oriented Programming (OOP)

- ▶ Software is built using **Objects**.
- ▶ Objects combine **data + functions**.
- ▶ Provides **Encapsulation**, improving security.
- ▶ Supports **Inheritance** for code reuse.
- ▶ Supports **Polymorphism** for flexibility.
- ▶ Closely models real-world systems.





Real-world Example Hospital Management System

POP

- ▶ Functions like addPatient(), billPatient(), searchPatient()
- ▶ Data shared globally
- ▶ Hard to maintain

OOP

- ▶ Objects: Patient, Doctor, Nurse, Hospital, Appointment
- ▶ Each object stores its own data securely
- ▶ Easy to extend and reuse



Real-world Example: Hospital Management System

Let's build a Hospital Management System using two different approaches.

Procedure-Oriented Programming (POP)



Design Approach

The system is divided into functions (procedures) that operate on data.

Example Functions

```
addPatient()  
billPatient()  
searchPatient()  
updateRecord()  
...
```



Data



Global Data
(Accessible by all functions)



Characteristics

- ✗ Functions act on data.
- ✗ Data is global and shared.
- ✗ Any function can access/modify data.
- ✗ Difficult to maintain as the system grows.
- ✗ Less secure (no data hiding).
- ✗ Poor code reusability.



Hard to Maintain

VS

Object-Oriented Programming (OOP)



Design Approach

The system is modeled using objects that combine data (attributes) and functions (methods).

Example Objects

Patient



Attributes

- id, name, age, address, ...

Methods

- getInfo()
- updateInfo()

Doctor



Attributes

- id, name, specialization, ...

Methods

- getSchedule()
- updateInfo()

Nurse



Attributes

- id, name, department, ...

Methods

- assignDuty()
- getInfo()

Hospital



Attributes

- name, address, departments, ...

Methods

- addDepartment()
- getDetails()

Appointment



Attributes

- id, date, time, patient, doctor, ...

Methods

- schedule()
- cancel()



Characteristics

- ✓ Data is encapsulated within objects.
- ✓ Each object controls its own data.
- ✓ Data is secure (data hiding).
- ✓ Easy to maintain and extend.
- ✓ Supports code reusability (inheritance).
- ✓ Models real-world entities.



Easy to Extend and Reuse

Key Difference



POP treats data as global and functions as primary.

VS

OOP treats objects as primary that combine data and behavior.

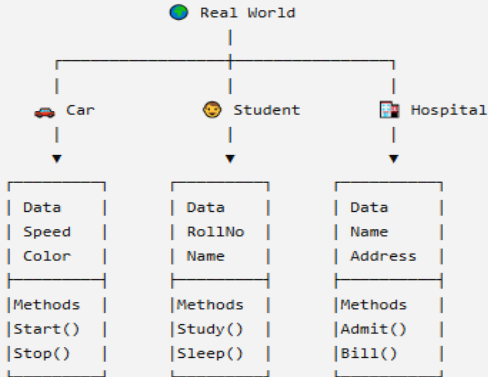




Definition of Object Orientation



WHAT IS OBJECT ORIENTATION?



Definition

- ▶ Object Orientation is a software development paradigm in which a program is organized as a collection of interacting **objects**, where each object contains both **data (attributes)** and **behavior (methods)**.
- ▶ An object-oriented system focuses on what an object is and what it can do, rather than simply describing a sequence of steps to perform a task.
- ▶ Object Orientation is a methodology for designing software by representing real-world entities as software objects that communicate with one another through messages.



What is an Object?

Object

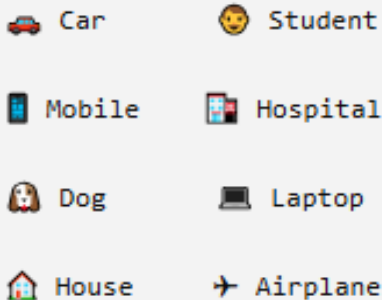
- ▶ Everything around us can be viewed as an Object.
- ▶ Every real-world entity can be represented as an Object.

Definition

An Object is an identifiable real-world entity that has Identity, State (Attributes), and Behavior (Methods).

Object = Identity + State (Attributes) + Behaviour (Methods)

OBJECTS AROUND US





What is an Object?

Example: Car Object

Identity

Registration
No. KA01AB1234



State (Attributes)

- Color : Blue
- Model : Honda City
- Year : 2024
- Fuel : Petrol
- Speed : 60 km/h

Behavior (Methods)

- ⚙ start()
- ⚙ stop()
- ⚙ accelerate()
- ⚙ brake()
- ⚙ turnLeft()
- ⚙ turnRight()



What is a Class?

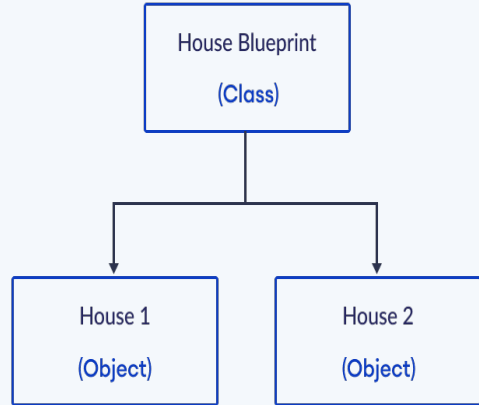


Why do we need Classes?

- ▶ Suppose a college has 5000 Students.
- ▶ Should we define each student separately?
- ▶ We create ONE blueprint called CLASS

Definition

- ▶ **A Class is a blueprint or template that defines the properties (attributes) and behaviors (methods) common to a group of similar objects**
- ▶ Objects are created from this blueprint.





Components of a Java Class





Fields, Methods and Constructors

Fields

- ▶ Store the state (data) of an object.
- ▶ Also called **instance variables**.
- ▶ Can be public, private, or protected.

Methods

- ▶ Define the behavior of an object.
- ▶ Can return a value or be void.

Constructors

- ▶ Initialize objects.
- ▶ Have the same name as the class.
- ▶ Invoked automatically using new.

```
1  class Student {  
2  
3      // Field  
4      String name;  
5  
6      // Constructor  
7      Student(String n){  
8          name = n;  
9      }  
10  
11     // Method  
12     void display(){  
13         System.out.println(name);  
14     }  
15 }
```



Blocks and Nested Classes

Blocks

► Static Block

- Executes once when the class is loaded.

► Instance Block

- Executes whenever an object is created.

Nested Classes

- Class defined inside another class.
- Can access members of the enclosing class.
- Used for better organization.

```
1  class Demo {  
2  
3      static {  
4          System.out.println("Static Block");  
5      }  
6  
7      {  
8          System.out.println("Instance Block");  
9      }  
10  
11     static class Inner {  
12         System.out.println("Inner Class");  
13     }  
14 }
```

Summary

A Java class is a blueprint that consists of **fields**, **methods**, **constructors**, **blocks**, and **nested classes/interfaces**, enabling data storage, behavior, object initialization, and code organization.



ASSESSMENT QUESTION



Q: Consider a **Student** object in a university management system.
What should be the **Identity**, **State (Attributes)** and **Methods (Behaviors)** of this object?

Student Object



HINTS



- Identity is always unique.
- State represents the data or information.
- Methods represent the actions or behaviors.



IDENTITY (Unique Identification)

What uniquely identifies a student?

- _____
- _____
- _____
- _____



STATE (Attributes)

What are the properties or data of a student?

- _____
- _____
- _____
- _____
- _____



METHODS (Behaviors)

What actions can a student perform?

- _____
- _____
- _____
- _____
- _____



Think about a real student and list the most relevant identity, state and methods.





Class and Object

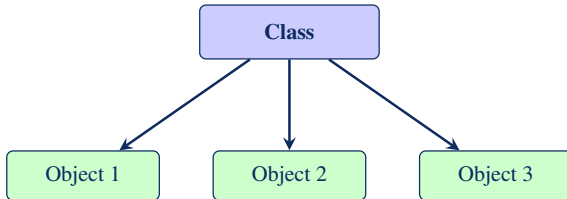
Class

- ▶ A class is a **blueprint** or template.
- ▶ It defines the data (fields) and behavior (methods).
- ▶ Objects are created from a class.

Object

- ▶ Instance of a class.
- ▶ Real-world entity.
- ▶ Runtime entity.
- ▶ Has **state** and **behavior**.

A class is a blueprint from which multiple objects are created.





Syntax

```
ClassName objName = new ClassName();
```

The new Keyword

- ▶ Allocates memory at runtime.
- ▶ Creates an object in the **Heap Memory**.
- ▶ Returns a reference to the object.

```
1  
2 Student s1 =  
3 new Student();  
4  
5 Student s2 =  
6 new Student();
```



Object and Class



ASSESSMENT QUESTION

Consider the **HOSPITAL MANAGEMENT SYSTEM**.

Below is the list of possible identities, attributes (state) and methods (behaviors).



Identify and group the appropriate **Identity**, **Attributes (State)** and **Methods (Behaviors)** for each object.

OBJECTS IN THE SYSTEM



Patient



Doctor



Nurse



Appointment



Billing



Hospital



Medicine

LIST OF ITEMS (Mixed List)

Patient ID

Name

Department

Specialization

Appointment ID

Medicine ID

Nurse ID

Doctor ID

Date & Time

Qualification

Symptoms

Bill ID

Hospital ID

Dose

Age

Room Number

Diagnosis

Address

Amount

Phone Number

StartShift()

Gender

Email

AppointmentDate()

RegisterPatient()

AssignDoctor()

GenerateBill()

EndShift()

AdmitPatient()

PrescribeMedicine()

CancelAppointment()

UpdateRecord()

ScheduleAppointment()

DischargePatient()

- Identity**
(Unique Identification)
- Attributes (State)**
(Properties / Data)
- Methods (Behaviors)**
(Actions / Functions)

YOUR TASK

Object	Patient	Doctor	Nurse	Appointment	Billing	Hospital	Medicine
Identity (Unique)	• _____	• _____	• _____	• _____	• _____	• _____	• _____
Attributes (State)	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____
Methods (Behaviors)	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____	• _____ • _____

Note: Some items may be used for more than one object.

Think carefully and place each item in the correct category.



Example: main() Inside the Class

```
1
2 class Main{
3
4     int id;
5     String name;
6
7     public static void main(String args[]){
8         Main s1 =
9         new Main();
10
11         System.out.println(s1.id);
12         System.out.println(s1.name);
13     }
14 }
```

Output

0
null

Explanation

- ▶ Object created using new.
- ▶ Fields are not initialized.
- ▶ Java assigns default values.



Example: main() Outside the Class



```
1
2 class Student{
3
4     int id;
5     String name;
6 }
7
8 class Main{
9     public static void main(String args[]){
10
11         Student s1 = new Student();
12         System.out.println(s1.id);
13         System.out.println(s1.name);
14     }
15 }
```

Output

0
null

Best Practice

Keep the main() method in a separate driver class.



Object Initialization

- ▶ Object initialization means assigning values to the object's fields.
- ▶ An object can be initialized in three ways:
 1. Using a **Reference Variable**
 2. Using a **Method**
 3. Using a **Constructor**



Initialization through Reference Variable

```
1 class Student{
2     int id;
3     String name;
4 }
5
6 class Main{
7     public static void main(String args[]){
8         Student s1=new Student();
9         s1.id=101;
10        s1.name="Sonoo";
11        System.out.println(s1.id+" "+s1.name);
12    }
13 }
```

Output

101 Sonoo

Explanation

- ▶ Create object using new.
- ▶ Assign values using the reference variable.
- ▶ Access fields using the dot operator (.).



Initialization through Method

```
1 class Student{
2     int rollno;
3     String name;
4     void insertRecord(int r,String n)
5     {   rollno=r;
6         name=n;
7     }
8     void display(){
9         System.out.println(rollno+" "+name);
10    }
11 }
12 class Main{
13     public static void main(String args[]){
14         Student s1=new Student();
15         s1.insertRecord(111,"Karan");
16         s1.display();
17     }}
```

Output

111 Karan

Advantages

- ▶ Reusable
- ▶ Cleaner code
- ▶ Easy to update multiple objects



Initialization through Constructor

```
1 class Student{
2     int id;
3     String name;
4     Student(int i,String n){
5         id=i;
6         name=n;
7     }
8     void display(){
9         System.out.println(id+" "+name);
10    }
11 }
12 class Main{
13     public static void main(String args[]){
14         Student s1= new Student(1,"John");
15         s1.display();
16     }
17 }
```

Output

1 John

Advantages

- ▶ Automatic initialization
- ▶ Cleaner code
- ▶ Preferred in real-world programs

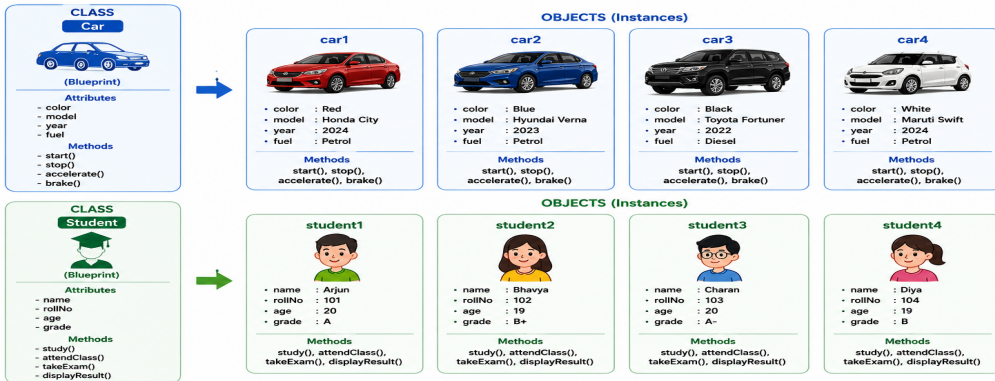
Note

A constructor is called automatically whenever an object is created using the new keyword.



CLASSES AND OBJECTS IN OOPS

A class is a blueprint. Objects are real-world instances of that blueprint.



Class is a template or blueprint that defines attributes and methods.
Objects are real-world entities created from a class.

■ Car Class → Car Objects
■ Student Class → Student Objects



Classes vs Objects

Class

- ▶ Blueprint
- ▶ Logical Entity
- ▶ No Memory
- ▶ Defines Properties
- ▶ One Class

Object

- ▶ Instance
- ▶ Physical Entity
- ▶ Occupies Memory
- ▶ Contains Actual Data
- ▶ Many Objects



Classes and Objects?



CLASSES AND OBJECTS – ASSESSMENT

B.Tech First Year



1. Identify Class and Object (Easy)

Fill the table with appropriate Class and Object.

Entity	Class	Object Example
Laptop	?	?
Student	?	?
Car	?	?
Mobile Phone	?	?
Bank Account	?	?



2. Fill the Missing Attributes

A Student object is shown below.

Student	Which of the following are attributes?
Name : _____	• Name
Roll No : _____	• Study()
Branch : _____	• Roll Number
CGPA : _____	• CGPA
	• AttendClass()
	• Branch

3. State vs Behavior

Classify the following into State (Attributes) and Behavior (Methods).

Student	State (Attributes)	Behavior (Methods)
Name	• _____	
Age	• _____	
Study()		• _____
Sleep()		• _____
Department		
Marks		
PlaySports()		• _____

4. Hospital Management System

Identify Classes, Objects, Attributes and Methods.

Class	Example Object	Attributes (State)	Methods (Behavior)

5. Guess the Class

Identify the class from the given objects.

Object 1	Object 2	Object 3
Name : Virat Jersey : 18	Name : Rohit Jersey : 45	Name : Bumrah Jersey : 93

Class Name : _____

6. Design Your Own Class

Design a Library Book class.

Attributes	Methods
• _____	• _____
• _____	• _____
• _____	• _____

7. Identify the Odd One

Which one is NOT an object?

A. Dell Laptop	C. Cat
B. Honda City	D. Samsung Galaxy S24

8. Match the Following

1. Student	•	A. Rotate()
2. Car	•	B. Call()
3. Mobile	•	C. Start()
4. Fan	•	D. Study()

9. Build the Class

Observe the smartphone and answer.

Class Name : _____
Five Attributes
1. _____
2. _____
3. _____
4. _____
5. _____
Five Methods
1. _____
2. _____
3. _____
4. _____
5. _____

10. Object Identity

	Suppose three students have exactly the same Name, Age, Branch and CGPA.
	Can they still be different objects?
	If yes, which attribute uniquely identifies them?

11. Complete the Diagram

Student (Class)
Object 1
Object 2
Object 3

Add 2 Attributes :	Add 2 Methods :
1. _____	1. _____
2. _____	2. _____

12. Think – Pair – Share Activity

Imagine you are designing an Online Food Delivery App.

In pairs, identify:

- 5 Classes
- 2 Objects for each class
- 3 Attributes per class
- 3 Methods per class



Think | Discuss | Write | Share

"In OOPs, we don't just write programs, we model the real world!"





Every object is unique, even if its data looks the same.

Definition

- ▶ **Object Identity** is the unique property that distinguishes one object from all other objects.
- ▶ In OOP, two objects may have the same state and behavior, but they are still different objects if they have different identities.



Every object is unique, even if its data looks the same.

Definition

- ▶ **Object Identity** is the unique property that distinguishes one object from all other objects.
- ▶ In OOP, two objects may have the same state and behavior, but they are still different objects if they have different identities.

Real-World Analogy

Consider **two identical laptops** of the exact same model, color, and specifications manufactured by the same brand.

- ▶ Their **state/attributes/data** is identical (same RAM, CPU, storage).
- ▶ Their **behavior/methods** is identical (they run the same OS, execute the same apps).
- ▶ Their **identity** is distinct (each has a unique hardware serial number, and physical existence). One laptop does not automatically become the other



Student Object 1

- ▶ Name : Rahul
- ▶ Age : 20
- ▶ Dept : CSE
- ▶ Roll No: 101

Student Object 2

- ▶ Name : Rahul
- ▶ Age : 20
- ▶ Dept : CSE
- ▶ Roll No: 102



Student Object 1

- ▶ Name : Rahul
- ▶ Age : 20
- ▶ Dept : CSE
- ▶ Roll No: 101

Student Object 2

- ▶ Name : Rahul
- ▶ Age : 20
- ▶ Dept : CSE
- ▶ Roll No: 102

Key Point

SameState $\neq$ SameObject

- ▶ A student may have the same name, age, and department as another student, but both are not the same student. Their Roll Number or Student ID uniquely identifies them.
- ▶ The **Student ID** uniquely identifies the student in the application's domain, while the **memory address** (or object reference) uniquely identifies the object during program execution.

Thank You

Keep learning, keep exploring, and keep coding!